Лабораторно упражнение 1 – II част

Тема: Мултикастинг на делегат

I. Теоретична част

1. Общи положения

Делегатите в език C# са типове, които представят референции към методи с определени параметри и конкретен тип на върнат резултат. Когато се инстанцира делегат (създава се променлива от този тип), тази инстанция се свързва с конкретен метод, който има същата сигнатура т.е. същите параметри и същият тип на върнат резултат. Тази инстанция се използва за извикване на метода.

Делегатите се използват за да бъдат предадени методи като аргументи на други методи. На практика манипулаторите на събития са методи, които се извикват чрез делегати.

2. Деклариране и инстанциране на делегати

2.1. Деклариране на делегат

Декларацията на делегат определя какви методи ще бъдат реферирани чрез делегата. Тези методи трябва да имат същата сигнатура като делегата.

Общият вид на декарацията на делегат е следната:

модификаторНаДостъп delegate тип ИмеДелегат(списък параметри);

Пример:

```
public delegate int MyDelegate (string s);
```

```
public delegate void Del(string message);
```

Веднъж деклариран, делегатът е тип, който може да бъде инстанциран.

2.2. Инстанциране на делегат

За да бъде инстанциран делегат, необходимо е да съществува метод, който да има същата сигнатура като делегата. Например:

```
// Create a method for a delegate.
public static void DelegateMethod(string message)
{
    Console.WriteLine(message);
}
```

Съответно, инстанцирането на делегата има следния синтаксис:

ИмеДелегат имеОбект = ИмеМетод;

Пример:

```
// Instantiate the delegate.
Del handler = DelegateMethod;
```

Извикването на делегат е чрез следния програмен ред:

```
// Call the delegate.
handler("Hello World");
```

Общият вид на извикване на делегат е:

имеОбектДелегат(списък параметри);

3. Примери

```
public class DelegateExamp
    {
        public delegate void addnum(int a, int b);
        public delegate void subnum(int a, int b);

        // method "sum"
        public void sum(int a, int b)
        {
            Console.WriteLine("({0} + {1}) = {2}", a,b, a + b);
        }

        // method "subtract"
        public void subtract(int a, int b)
        {
            Console.WriteLine("({0} - {1}) = {2}", a,b,a - b);
        }

    }


DelegateExamp exm = new DelegateExamp();
```

```
DelegateExamp.addnum del_obj1 = new DelegateExamp.addnum(exm.sum);
DelegateExamp.subnum del_obj2 = new DelegateExamp.subnum(exm.subtract);

del_obj1(100, 40);
del_obj2(100, 60);

del_obj1(100, 40);
del_obj2(100, 60);

del_obj1.Invoke(20, 30);
```

4. Последователно извикване на методи чрез делегат (Multicasting на делегат)

Multicasting на делегат е разширение на нормалния делегат, наричан понякога single cast delegate т.е. делегатът извиква само един метод. Това позволява да бъдат достъпени няколко метода с един делегат и съответно извикани последователно с едно обръщение. Делегатът сочи списък от методи и при обръщение към него, последователно се извикват всички методи в списъка. Обръщението към методите е от тип FIFO.

Добавянето на метод към списъка е чрез оператори + или +=. Съответно, оператори – и -= премахват метод от списъка.

При мултикастинга на делегат е необходимо да няма тип на върнат резултат т.е. делегатът и съответно методите да са от тип ***void***. В противен случай, ще бъде върната стойността на последния извикан метод.

Пример:

```
class rectangle
        {

            public delegate void rectDelegate(double height,
                                              double width);


            public void area(double height, double width)
            {
                Console.WriteLine("Area is: {0}", (width * height));
            }

            public void perimeter(double height, double width)
            {
                Console.WriteLine("Perimeter is: {0} ", 2 * (width +
height));
            }
        }
```

```
            rectangle rect = new rectangle();

            rectangle.rectDelegate rectdele = new
rectangle.rectDelegate(rect.area);

            rectdele += rect.perimeter;


            rectdele.Invoke(6.3, 4.2);
            Console.WriteLine();

            rectdele.Invoke(16.3, 10.3);
```

II. Задачи за изпълнение

1. Да се създаде клас ExampleOperation, който съдържа методи за сумиране, изваждане, умножение и деление на две дробни числа. Методите да връщат резултат от изпълнението на операцията. Да се декларират делегат, чрез който да прави обръщение към различните операции. Да се създаде демонстрационна работа с класа, която симулира работата на калкулатор.
2. Да се модифицира програмния код в задача 1 като методите не връщат резултат, а извеждат на екрана резултата от изпълнението на операцията. Да се създаде демонстрационна програма, която последователно извиква методите за събиране, изваждане, умножение и деление чрез използване на multicasting.